Data Mining Assignment

Lab 1

Done by:

Shadi Reda Elattar - 8180
Omar Abourida - 8192

PCA Methodology:

Objective To reduce the high-dimensional feature space of the dataset while retaining 95% of the original variance, preparing the data for efficient downstream classification without compromising structural integrity or introducing data leakage.

1. Data Segregation and Leakage Prevention Prior to any transformation, the raw dataset was divided using a 50/50 stratified train-test split. The stratification ensures that the class distribution remains consistent across both subsets. Splitting the data as the very first step strictly isolates the test set, ensuring the PCA model has no exposure to unseen data, thereby preventing data leakage.

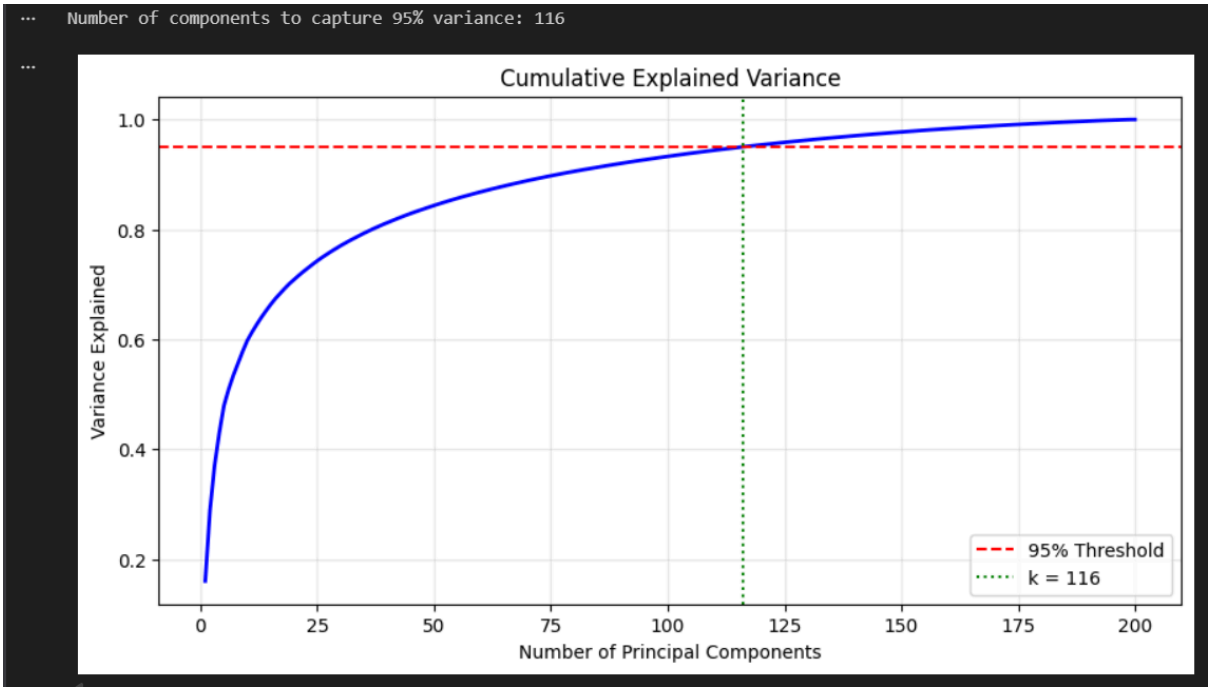
```
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.5, stratify=y, random_state=42
    )
```

2. Consistent Standardization To ensure all features contribute equally to the variance calculations, Z-score normalization was applied. The mean and standard deviation were calculated **exclusively from the training set**. These training statistics were then used to normalize both the training and test sets, ensuring the test data was mapped perfectly into the exact same coordinate space established by the training data.

```
# 1. Standardization
features_mean = np.mean(X_train, axis=0)
features_std = np.std(X_train, axis=0)
X_train_normalized = (X_train - features_mean) / features_std
X_test_normalized = (X_test - features_mean) / features_std # no
```

3. Dual PCA Implementation (Gram Matrix Method) Given the high dimensionality of the data ($D \gg N$), calculating the standard $D \times D$ feature covariance matrix was computationally unfeasible. Instead, the Dual PCA trick was utilized by calculating the $N \times N$ sample covariance matrix ($X \cdot X^T$). This drastically reduced memory consumption while preserving the core mathematical relationships of the data.

4. Eigendecomposition and Variance Thresholding Eigenvalues and eigenvectors were extracted from the sample covariance matrix and sorted in descending order to prioritize the components capturing the maximum variance. To determine the optimal number of principal components (K), a cumulative explained variance ratio was calculated. The threshold was set to strictly retain **95%** of the dataset's total variance, dynamically calculating the required K.



5. Feature Space Recovery and Final Projection Because the eigendecomposition yielded "sample" eigenvectors, a linear transformation was required to recover the true "feature" eigenvectors. This was achieved by multiplying the transposed normalized training data by the sample eigenvectors ($X^T \cdot V$). These recovered feature eigenvectors were mathematically normalized to unit length ($||v|| = 1$) and truncated to the top K components (W_k).

Finally, both the normalized training and test datasets were projected onto this K-dimensional subspace via dot product mapping, successfully completing the dimensionality reduction pipeline.

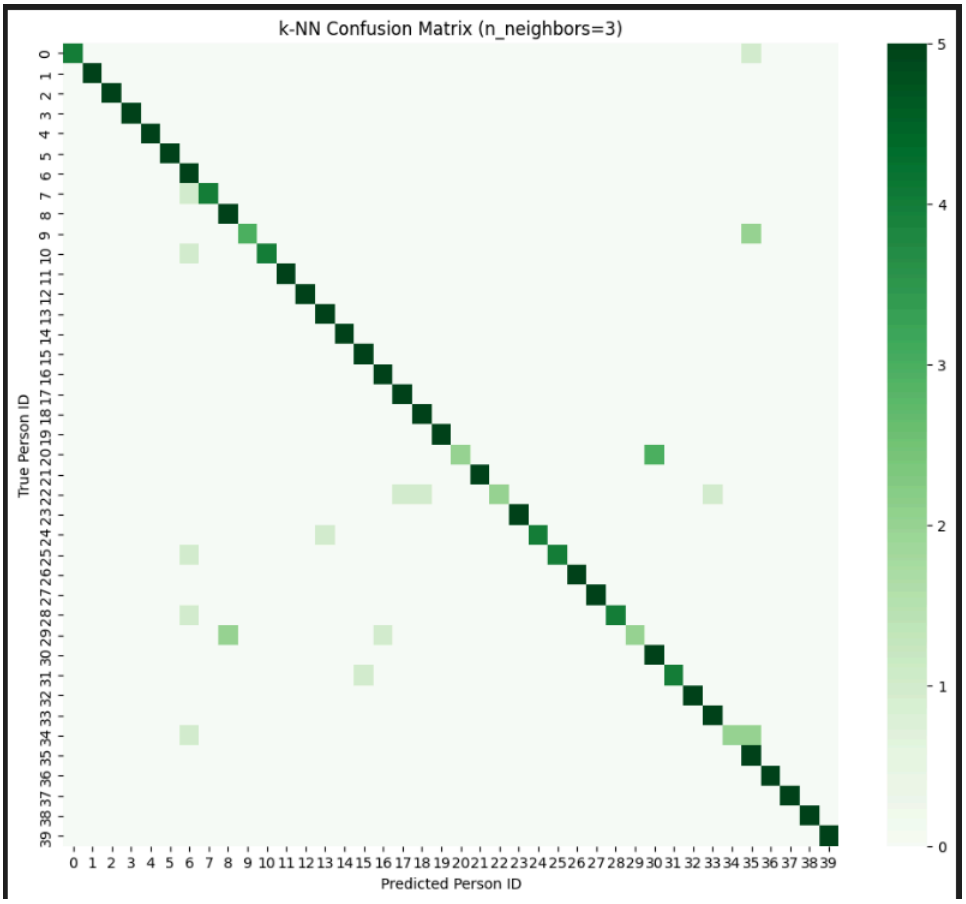
Results – with and without PCA comparison

With PCA:

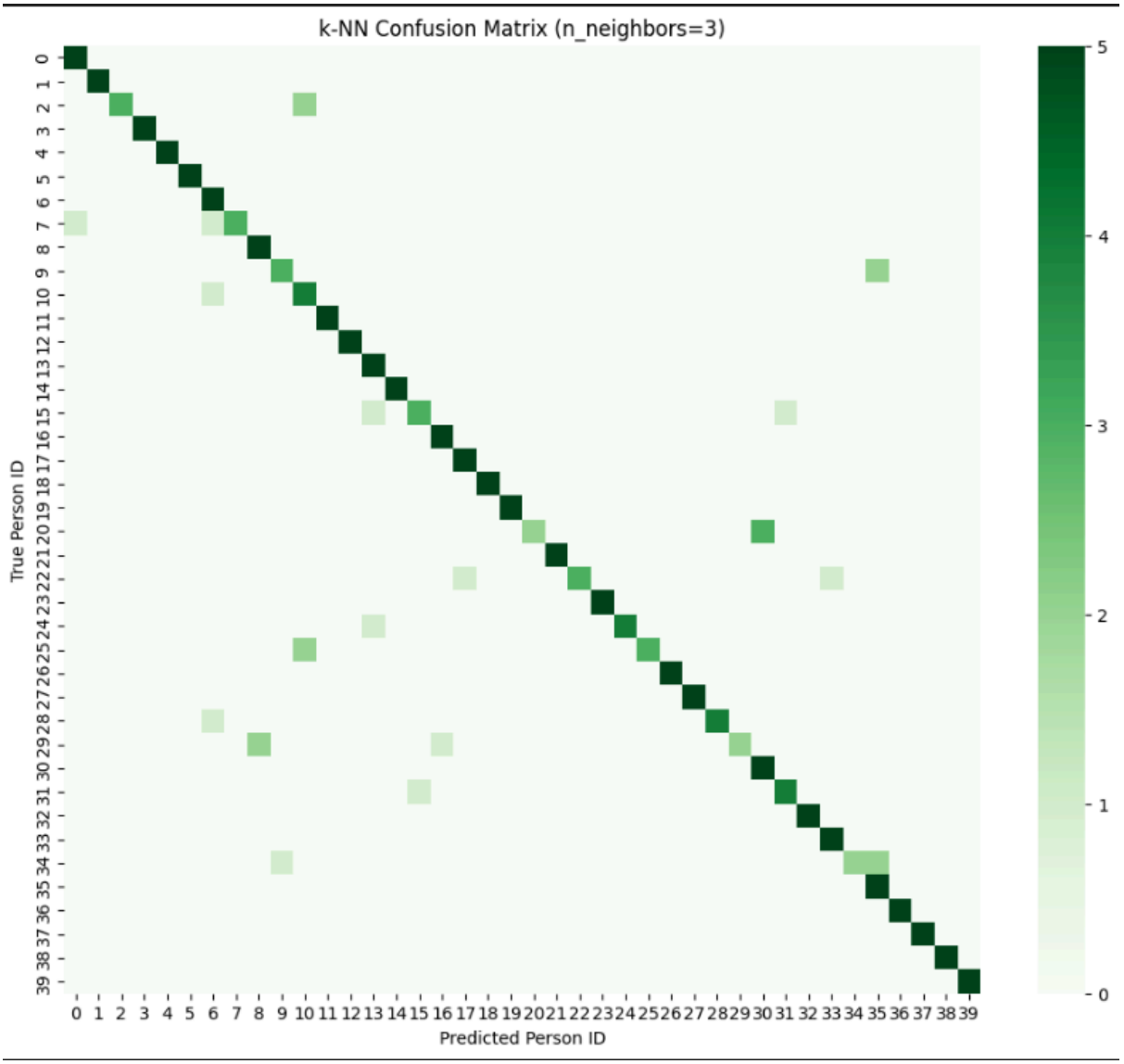
```
k-NN Accuracy (n_neighbors=3): 0.8950

Detailed Performance Metrics (Precision, Recall, F1-Score):
              precision    recall  f1-score   support

           0       1.00      0.80      0.89         5
           1       1.00      1.00      1.00         5
           2       1.00      1.00      1.00         5
           3       1.00      1.00      1.00         5
           4       1.00      1.00      1.00         5
           5       1.00      1.00      1.00         5
           6       0.50      1.00      0.67         5
           7       1.00      0.80      0.89         5
           8       0.71      1.00      0.83         5
           9       1.00      0.60      0.75         5
          10       1.00      0.80      0.89         5
          11       1.00      1.00      1.00         5
          12       1.00      1.00      1.00         5
          13       0.83      1.00      0.91         5
          14       1.00      1.00      1.00         5
          15       0.83      1.00      0.91         5
          16       0.83      1.00      0.91         5
          17       0.83      1.00      0.91         5
          18       0.83      1.00      0.91         5
          19       1.00      1.00      1.00         5
...
    accuracy                           0.90       200
   macro avg       0.93      0.89      0.89       200
weighted avg       0.93      0.90      0.89       200
```


k-NN Confusion Matrix (n_neighbors=3)
True Person ID
Predicted Person ID
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39
0 1 2 3 4 5

Without PCA:


k-NN Confusion Matrix (n_neighbors=3)
True Person ID
Predicted Person ID
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39
0 1 2 3 4 5

```
k-NN Accuracy (n_neighbors=3): 0.8750

Detailed Performance Metrics (Precision, Recall, F1-Score
              precision    recall  f1-score   support

           0       0.83      1.00      0.91         5
           1       1.00      1.00      1.00         5
           2       1.00      0.60      0.75         5
           3       1.00      1.00      1.00         5
           4       1.00      1.00      1.00         5
           5       1.00      1.00      1.00         5
           6       0.62      1.00      0.77         5
           7       1.00      0.60      0.75         5
           8       0.71      1.00      0.83         5
           9       0.75      0.60      0.67         5
          10       0.50      0.80      0.62         5
          11       1.00      1.00      1.00         5
          12       1.00      1.00      1.00         5
          13       0.71      1.00      0.83         5
          14       1.00      1.00      1.00         5
          15       0.75      0.60      0.67         5
          16       0.83      1.00      0.91         5
          17       0.83      1.00      0.91         5
          18       1.00      1.00      1.00         5
          19       1.00      1.00      1.00         5
...
    accuracy                           0.88       200
   macro avg       0.91      0.88      0.87       200
weighted avg       0.91      0.88      0.87       200
```

- **Without PCA (Raw Data):** The k-NN model achieved an overall accuracy of **87.5%**, with macro-average Precision, Recall, and F1-scores hovering around **0.87** to **0.91**.
- **With PCA (Reduced Data):** The accuracy jumped up to **89.5%**, and the macro-average metrics improved to **0.89** to **0.93**.

<u>Why?</u>

A. Overcoming the "Curse of Dimensionality"

The original dataset has massive dimensionality (10,304 features per image). In extremely high-dimensional spaces, a mathematical phenomenon occurs where the Euclidean distance between *any* two points starts to look roughly the same. Because k-NN relies entirely on calculating these straight-line Euclidean distances, it gets "confused" by the sheer volume of dimensions. By reducing the space to just K dimensions, the distances between the data points become mathematically meaningful again, allowing k-NN to easily find the true nearest neighbors.

B. PCA as a Noise Filter

When we chose a K that captured exactly 95% of the variance, we intentionally threw away the bottom 5%. In image datasets like this, that bottom 5% usually doesn't contain important structural features (like eyes, noses, or jawlines). Instead, it contains **noise**: slight lighting variations, camera static, or irrelevant background pixels. By discarding that 5%, we effectively filtered out the noise. The model with PCA was evaluating clean, core "eigenfaces," whereas the raw model was getting distracted by irrelevant pixel variations.

C. Computational Efficiency

While the execution time on a tiny dataset of 200 test images might look similar in a Jupyter notebook, the theoretical implications are massive. The PCA model is calculating distances using a fraction of the features. In a real-world, large-scale facial recognition system, the PCA pipeline would save massive amounts of memory (RAM) and severely cut down on prediction latency.